



Instituto Politécnico Nacional Escuela Superior de Cómputo



Ingeniería en Inteligencia Artificial

Grupo: 4BV1

Unidad de aprendizaje: Procesamiento Digital de Imágenes

Caso #1

Profesor: Saul de la O Torres

Alumno:

Velázquez Arrieta Eduardo Uriel

índice de contenido

índice de contenido	1
Índice de imágenes	1
Introducción	2
Desarrollo	2
Sal y pimienta	2
Ruido gaussiano	2
Vista	3
Controlador	3
Modelo	4
Main	5
Conclusiones	6

Índice de imágenes

Imagen 1. Fórmula para sal y pimienta	2
Imagen 2. Fórmula para ruido gaussiano	2
Imagen 3. Código de vista.	3
Imagen 4. Código de controlador.	4
Imagen 5. Código de sal y pimienta.	5
Imagen 6. Código de ruido gaussiano.	5
Imagen 7. Código de main	6

Introducción

El documento actual es un caso de estudio para la materia de Procesamiento Digital de Imágenes (PDI) y la sección donde el cursor está posicionado es la [Introducción](#). El código proporcionado en la solicitud implementa métodos para añadir **Ruido Sal y Pimienta** y **Ruido Gaussiano** a una imagen digital.

Por lo tanto, la introducción debe contextualizar la importancia de manejar el ruido en PDI y presentar el objetivo del caso práctico, que es implementar y demostrar la aplicación de estos modelos de ruido.

En el ámbito del Procesamiento Digital de Imágenes (PDI), la calidad de una imagen capturada o transmitida puede verse seriamente afectada por la presencia de **ruido**. Este ruido se manifiesta como variaciones no deseadas en la intensidad de los píxeles, lo que degrada la información visual y dificulta las tareas de análisis e interpretación. Comprender, modelar y simular diferentes tipos de ruido es fundamental para el desarrollo de algoritmos de filtrado y restauración efectivos. El presente caso de estudio se centra en la implementación y aplicación de dos modelos de ruido ampliamente estudiados: el **Ruido Sal y Pimienta** (o ruido impulsivo) y el **Ruido Gaussiano**. Estos modelos permiten simular condiciones reales de degradación de la imagen, sentando las bases para evaluar el desempeño de técnicas posteriores de mejora. El objetivo principal es desarrollar una implementación práctica en Java que permita generar estas alteraciones en imágenes digitales y analizar sus efectos visuales.

Desarrollo

Sal y pimienta

Para sal y pimienta ocupamos la fórmula

$$P(z) = \begin{cases} p & \text{si } z = 0 \\ p & \text{si } z = 255 \\ 1 - 2p & \text{si } z = I_{orig} \end{cases}$$

Imagen 1. Fórmula para sal y pimienta

Ruido gaussiano

Para aplicar ruido gaussiano se aplica la siguiente fórmula

$$C'(x, y) = \text{clamp} \left(C(x, y) + \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{\eta^2}{2\sigma^2}} \right)$$

Imagen 2. Fórmula para ruido gaussiano

Vista

Es el componente de la **Interfaz de Usuario (UI)** dentro de una arquitectura **Modelo-Vista-Controlador (MVC)**, utilizada para la manipulación de imágenes y la aplicación de ruido (Sal y Pimienta, Gaussiano).

La clase Vista es responsable de:

- Mostrar la imagen.
- Capturar las interacciones del usuario (botones, sliders).
- Comunicar estas interacciones al Controlador.
- Recibir la imagen procesada del Controlador y mostrarla.

```
public Vista() {
    setTitle("Control de Ruido en Imagen - MVC");
    setSize(800, 600);
    setDefaultCloseOperation(EXIT_ON_CLOSE);
    setLayout(new BorderLayout());

    labelImagen = new JLabel();
    labelImagen.setHorizontalAlignment(JLabel.CENTER);
    add(new JScrollPane(labelImagen), BorderLayout.CENTER);

    JPanel panelPrincipalControl = new JPanel();
    panelPrincipalControl.setLayout(new BoxLayout(panelPrincipalControl, BoxLayout.Y_AXIS));
    panelPrincipalControl.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

    // 1. Fila de Carga y Restaurar
    JPanel panelBotones = new JPanel(new FlowLayout(FlowLayout.LEFT));
    JButton btnCargar = new JButton("Cargar Imagen");
    btnRestaurar = new JButton("Ver Original");
    btnRestaurar.setEnabled(false); // Deshabilitado hasta que haya imagen
    panelBotones.add(btnCargar);
    panelBotones.add(btnRestaurar);
    panelPrincipalControl.add(panelBotones);

    // Separador
    panelPrincipalControl.add(new JSeparator());

    // 2. Panel de Deslizadores (GridLayout para orden)
    JPanel panelSliders = new JPanel(new GridLayout(2, 3, 10, 5));
    panelSliders.setBorder(BorderFactory.createTitledBorder("Controles de Ruido (sobre original)"));

    // -- Controles Sal y Pimienta --
    JLabel lblSp = new JLabel("Ruido S&P (Probabilidad %):");
    // Slider de 0 a 100 representing 0.0 a 0.2 de probabilidad aprox
    sliderSalPimienta = new JSlider(JSlider.HORIZONTAL, 0, 100, 0);
    sliderSalPimienta.setMajorTickSpacing(20);
    sliderSalPimienta.setPaintTicks(true);
    sliderSalPimienta.setEnabled(false);
    lblValSp = new JLabel("0%");

    panelSliders.add(lblSp);
    panelSliders.add(sliderSalPimienta);
    panelSliders.add(lblValSp);
}
```

Imagen 3. Código de vista.

Controlador

Actúa como el intermediario central en la arquitectura Modelo-Vista-Controlador (MVC) del proyecto. Su función principal es manejar la lógica de interacción entre la interfaz de usuario (Vista) y la manipulación de datos/imágenes (Modelo).

Funciones Clave:

- Conexión MVC: Inicializa y mantiene referencias a los objetos Modelo y Vista.
- Inicialización (iniciar): Establece el propio Controlador dentro de la Vista, permitiendo a la UI notificarle sobre las acciones del usuario.
- Carga de Imagen (cargarImagen): Lee una imagen de un archivo usando ImageIO, la almacena en la Vista y la prepara para su procesamiento.
- Manejo de Ruido Sal y Pimienta (actualizarRuidoSalPimienta):
- Recibe el valor de un slider de la interfaz.
- Convierte este valor a un parámetro de probabilidad (tasa de ruido).
- Llama al método correspondiente en el Modelo para aplicar el ruido.
- Ordena a la Vista mostrar la imagen resultante.
- Manejo de Ruido Gaussiano (actualizarRuidoGaussiano):
- Recibe el valor de un slider de la interfaz (interpretado como el parámetro sigma).
- Llama al método correspondiente en el Modelo para aplicar el ruido gaussiano.
- Ordena a la Vista mostrar la imagen procesada.

```

public Controlador(Modelo modelo, Vista vista) {
    this.modelo = modelo;
    this.vista = vista;
}

public void iniciar() {
    vista.setControlador(this);
}

public void cargarImagen(String ruta) {
    try {
        BufferedImage img = ImageIO.read(new File(ruta));
        // Guardamos la original y mostramos la original inicialmente
        vista.setImagenOriginal(img);
    } catch (Exception e) {
        System.out.println("Error al cargar imagen: " + e.getMessage());
    }
}

// Metodo para cuando cambia el slider de Sal y Pimienta
public void actualizarRuidoSalPimienta(BufferedImage imgOriginal, int valorSlider) {
    if (imgOriginal == null) return;
    // Convertimos el valor del slider (0-100) a probabilidad (0.0 - 0.2)
    double probabilidad = valorSlider / 500.0;
    BufferedImage resultado = modelo.agregarRuidoSalPimienta(imgOriginal, probabilidad);
    vista.mostrarImagenProcesada(resultado);
}

// Metodo para cuando cambia el slider Gaussiano
public void actualizarRuidoGaussiano(BufferedImage imgOriginal, int valorSlider) {
    if (imgOriginal == null) return;
    // El slider va de 0 a 100, que es un buen rango para sigma
    double sigma = (double) valorSlider;
    BufferedImage resultado = modelo.agregarRuidoGaussiano(imgOriginal, sigma);
    vista.mostrarImagenProcesada(resultado);
}
}

```

Imagen 4. Código de controlador.

Modelo

Para el modelo simplemente aplicamos la fórmula de sal y pimienta respectivamente para cada método respectivamente.

```

public BufferedImage agregarRuidoSalPimienta(BufferedImage imagen, double probabilidad) {
    int width = imagen.getWidth();
    int height = imagen.getHeight();
    BufferedImage salida = new BufferedImage(width, height, imagen.getType());

    Random rand = new Random();

    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {

            double r = rand.nextDouble();

            if (r < probabilidad) {
                salida.setRGB(x, y, 0x000000); // negro
            } else if (r > 1 - probabilidad) {
                salida.setRGB(x, y, 0xFFFFFF); // blanco
            } else {
                salida.setRGB(x, y, imagen.getRGB(x, y));
            }
        }
    }
    return salida;
}

```

Imagen 5. Código de sal y pimienta.

```

public BufferedImage agregarRuidoGaussiano(BufferedImage imagen, double sigma) {
    int width = imagen.getWidth();
    int height = imagen.getHeight();
    BufferedImage salida = new BufferedImage(width, height, imagen.getType());

    Random rand = new Random();

    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {

            int rgb = imagen.getRGB(x, y);

            int r = (rgb >> 16) & 0xff;
            int g = (rgb >> 8) & 0xff;
            int b = rgb & 0xff;

            r = clamp((int) (r + rand.nextGaussian() * sigma));
            g = clamp((int) (g + rand.nextGaussian() * sigma));
            b = clamp((int) (b + rand.nextGaussian() * sigma));

            int nuevoRGB = (r << 16) | (g << 8) | b;
            salida.setRGB(x, y, nuevoRGB);
        }
    }
    return salida;
}

```

Imagen 6. Código de ruido gaussiano.

Main

Se encarga de inicializar todos los bloques de código.

```

public class Main {
    public static void main(String[] args) {

        Modelo modelo = new Modelo();
        Vista vista = new Vista();
        Controlador controlador = new Controlador(modelo, vista);

        controlador.iniciar();
        vista.setVisible(true);
    }
}

```

Imagen 7. Código de main

Conclusiones

Tanto el ruido gaussiano como el ruido sal y pimienta son operaciones relativamente sencillas de aplicar a imágenes, pero su simulación es crucial en el campo del Procesamiento Digital de Imágenes (PDI). Estos modelos de degradación nos permiten evaluar y construir algoritmos robustos destinados a la mejora y restauración de imágenes que han sufrido daños, ya sea durante la captura, transmisión, conversión (análoga a digital) o simplemente por el paso del tiempo y el almacenamiento.

Específicamente, el Ruido Sal y Pimienta nos ayuda a probar la efectividad de filtros no lineales, como el filtro de la mediana, mientras que el Ruido Gaussiano es fundamental para calibrar filtros lineales y no lineales diseñados para manejar variaciones estocásticas. Este caso práctico demostró que, a través de una arquitectura MVC, es posible modular la simulación del ruido, separando la lógica de negocio (Modelo) de la interacción del usuario (Vista y Controlador), resultando en una herramienta flexible y escalable para futuros experimentos de restauración de imágenes. La implementación exitosa de ambos modelos sienta las bases para el estudio y desarrollo de técnicas avanzadas de eliminación de ruido.